AI Assisted Coding

Lab ASSIGNMENT 11.1

K.Harish

2303A51858

B13

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty

methods.

Sample Input Code:

class Stack:

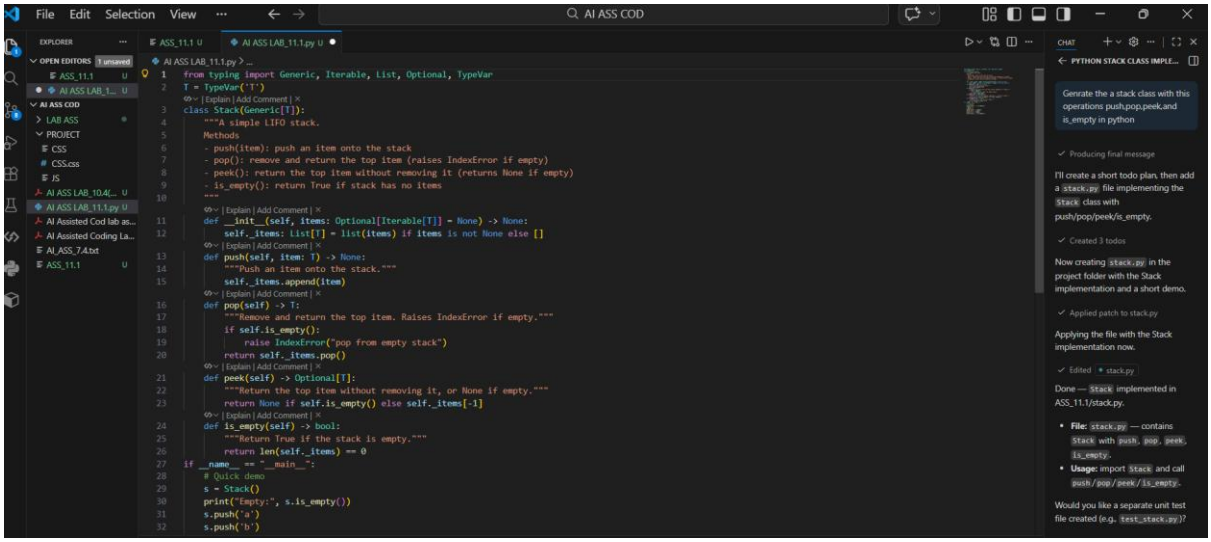
pass

Expected Output:

• A functional stack implementation with all required methods and

docstrings.

PROMPT: Genrate the a stack class with this operations push,pop,peek,and is_empty in python



OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell

PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI
 ASS COD/AI ASS LAB_11.1.py"
 ASS COD/AI ASS LAB_11.1.py"
Empty: True
Peek: b
Pop: b
Pop: a
Empty: True
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

PROMPT: Generate queue using python list and perform its operations

```python
from typing import Generic, Iterable, List, Optional, TypeVar
T = TypeVar('T')
class Queue(Generic[T]):
    """A simple FIFO queue implemented using Python list.
    Methods
    - enqueue(item): add item to the back of the queue
    - dequeue(): remove and return the front item (raises IndexError if empty)
    - peek(): return the front item without removing it (returns None if empty)
    - is_empty(): return True if queue has no items
    - size(): return number of items in the queue
    """
    def __init__(self, items: Optional[Iterable[T]] = None) -> None:
        self._items: List[T] = list(items) if items is not None else []
    def enqueue(self, item: T) -> None:
        """Add an item to the back of the queue."""
        self._items.append(item)
    def dequeue(self) -> T:
        """Remove and return the front item. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self._items.pop(0)
    def peek(self) -> Optional[T]:
        """Return the front item without removing it, or None if empty."""
        return None if self.is_empty() else self._items[0]
    def is_empty(self) -> bool:
        """Return True if the queue is empty."""
        return len(self._items) == 0
    def size(self) -> int:
        """Return number of items in the queue."""
        return len(self._items)
if __name__ == "__main__":
    # Quick demo
    q = Queue[int]()
    print("Empty:", q.is_empty())
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    print("Peek/front:", q.peek())
    print("Size:", q.size())
    print("Dequeue:", q.dequeue())
    print("Dequeue:", q.dequeue())
    print("Dequeue:", q.dequeue())
    print("Empty:", q.is_empty())
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell

PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI
 ASS COD/AI ASS LAB_11.1.py"
Empty: True
Peek/front: 10
Size: 3
Dequeue: 10
Dequeue: 20
Dequeue: 30
Empty: True
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

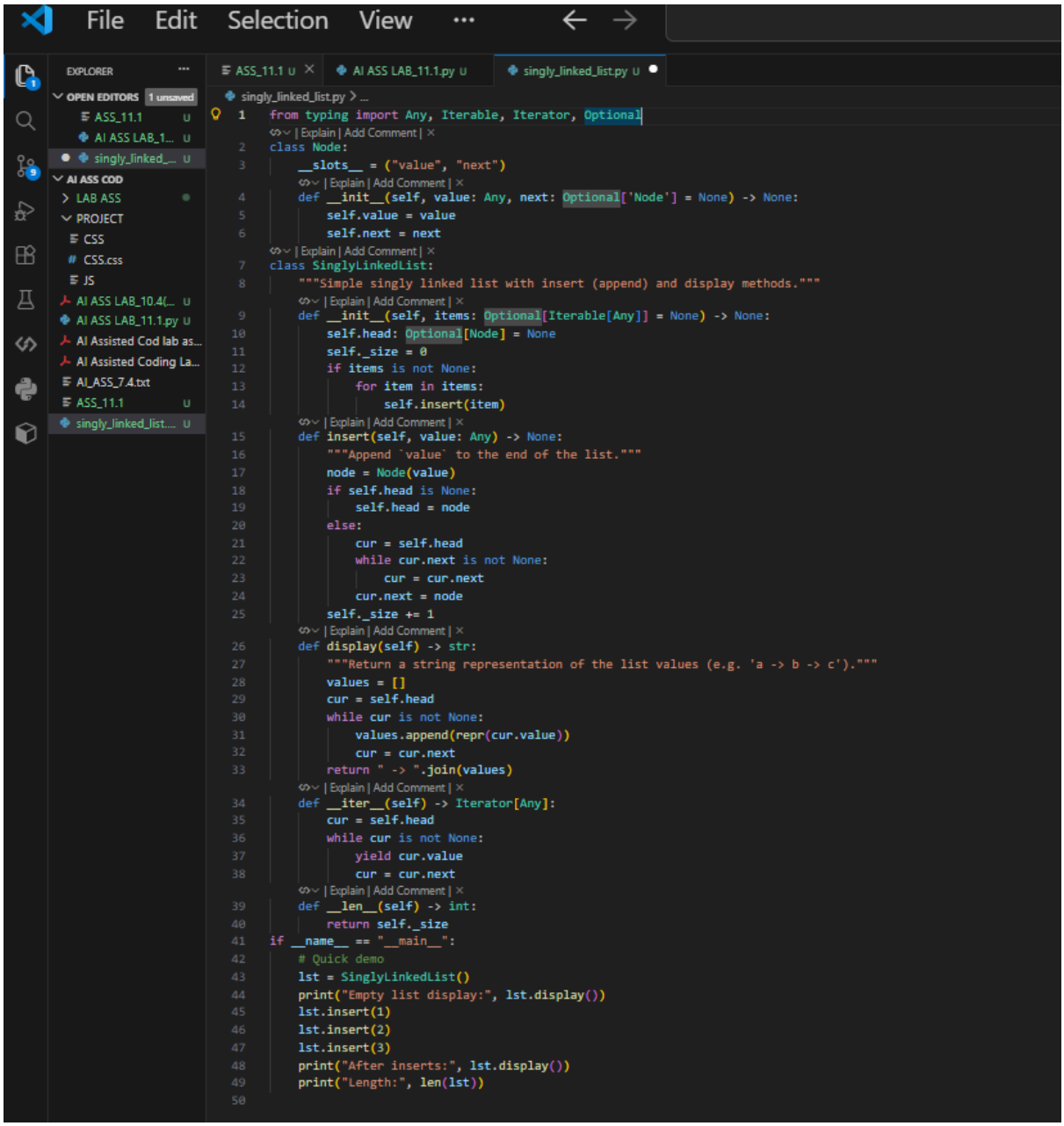
class Node:

pass

class LinkedList:

pass

PROMPT: Generate the singly linked list with insert and display methods



OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/singly_linked_list.py"
Empty list display:
After inserts: 1 -> 2 -> 3
Length: 3
After inserts: 1 -> 2 -> 3
Length: 3
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

PROMPT: Generate the Binary search tree with insert and traverse in-order method

ASS_11.1 U | AI ASS LAB_11.1.py U | singly_linked_list.py U | binary_search_tree.py U

binary_search_tree.py > BinarySearchTree > traverse_in_order

```python
from typing import Any, Iterator, List, Optional
class Node:
    __slots__ = ("value", "left", "right")
    def __init__(self, value: Any) -> None:
        self.value = value
        self.left: Optional[Node] = None
        self.right: Optional[Node] = None
class BinarySearchTree:
    """A simple binary search tree with insert and in-order traversal."""
    def __init__(self) -> None:
        self.root: Optional[Node] = None
    def insert(self, value: Any) -> None:
        """Insert `value` into the BST. Duplicates go to the right subtree."""
        if self.root is None:
            self.root = Node(value)
            return
        cur = self.root
        while True:
            if value < cur.value:
                if cur.left is None:
                    cur.left = Node(value)
                    return
                cur = cur.left
            else:
                if cur.right is None:
                    cur.right = Node(value)
                    return
                cur = cur.right
    def inorder(self) -> List[Any]:
        """Return a list of values from an in-order traversal."""
        out: List[Any] = []
        def _inorder(node: Optional[Node]) -> None:
            if node is None:
                return
            _inorder(node.left)
            out.append(node.value)
            _inorder(node.right)
        _inorder(self.root)
        return out
    def traverse_in_order(self) -> Iterator[Any]:
        """Yield values in in-order using an iterative generator."""
        stack: List[Node] = []
        cur = self.root
        while stack or cur is not None:
            while cur is not None:
                stack.append(cur)
                cur = cur.left
            cur = stack.pop()
            yield cur.value
            cur = cur.right
if __name__ == "__main__":
    bst = BinarySearchTree()
    for v in [7, 3, 9, 1, 5, 8, 10]:
        bst.insert(v)
    print("In-order (list):", bst.inorder())
    print("In-order (generator):", list(bst.traverse_in_order()))
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd
ktop/AI ASS COD/binary_search_tree.py"
In-order (list): [1, 3, 5, 7, 8, 9, 10]
In-order (generator): [1, 3, 5, 7, 8, 9, 10]
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

PROMPT: Generate Hash table with basic insert, search and delete methods

ASS_11.1 U | AI ASS LAB_11.1.py U | singly_linked_list.py U | binary_search_tree.py U | hash_table.py 6, U

hash_table.py > ...

```python
from typing import Any, List, Optional, Tuplclass HashTable:
    """A simple hash table using separate chaining.
    Methods
    - insert(key, value): store the value under key
    - search(key): return the value for key or None
    - delete(key): remove the key and return True if removed
    """
    | Explain | Add Comment | ×
    def __init__(self, capacity: int = 8) -> None:
        if capacity <= 0:
            raise ValueError("capacity must be > 0")
        self._capacity = capacity
        self._buckets: List[List[Tuple[Any, Any]]] = [[] for _ in range(capacity)]
        self._size = 0
    | Explain | Add Comment | ×
    def _bucket_index(self, key: Any) -> int:
        return hash(key) % self._capacity
    | Explain | Add Comment | ×
    def insert(self, key: Any, value: Any) -> None:
        """Insert or update the key with the given value."""
        idx = self._bucket_index(key)
        bucket = self._buckets[idx]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
        self._size += 1
    | Explain | Add Comment | ×
    def search(self, key: Any) -> Optional[Any]:
        """Return the value for key, or None if not found."""
        idx = self._bucket_index(key)
        bucket = self._buckets[idx]
        for k, v in bucket:
            if k == key:
                return v
        return None
    | Explain | Add Comment | ×
    def delete(self, key: Any) -> bool:
        """Delete key from the table. Return True if deleted, False if not found."""
        idx = self._bucket_index(key)
        bucket = self._buckets[idx]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self._size -= 1
                return True
        return False
    | Explain | Add Comment | ×
    def __len__(self) -> int:
        return self._size
    | Explain | Add Comment | ×
    def __repr__(self) -> str:
        items = []
        for bucket in self._buckets:
            for k, v in bucket:
                items.append(f"{k!r}: {v!r}")
        return "{" + ", ".join(items) + "}"
if __name__ == "__main__":
    # Quick demo
    ht = HashTable()
    ht.insert('apple', 3)
    ht.insert('banana', 5)
    ht.insert('orange', 2)
    print('Table:', ht)
    print("Search 'banana':", ht.search('banana'))
    print("Search 'grape':", ht.search('grape'))
    print("Delete 'orange':", ht.delete('orange'))
    print('Table after delete:', ht)
    print('Length:', len(ht))
```

OUTPUT:

```
td63/OneDrive/Desktop/AI ASS COD/hash_table.py
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhj
td63/OneDrive/Desktop/AI ASS COD/hash_table.py"
Table: {'apple': 3, 'banana': 5, 'orange': 2}
Search 'banana': 5
Search 'grape': None
Delete 'orange': True
Table after delete: {'apple': 3, 'banana': 5}
Length: 2
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

PROMPT: Generate a graph using an adjacency list

graphs.py > ...

```python
class Graph:
    - display()
    - bfs(start)
    - dfs_recursive(start)
    - dfs_iterative(start)
    """
    | Explain | Add Comment |
    def __init__(self, directed: bool = False) -> None:
        self.adj: Dict[Any, Set[Any]] = {}
        self.directed = directed
    | Explain | Add Comment |
    def add_vertex(self, v: Any) -> None:
        if v not in self.adj:
            self.adj[v] = set()
    | Explain | Add Comment |
    def add_edge(self, u: Any, v: Any) -> None:
        self.add_vertex(u)
        self.add_vertex(v)
        self.adj[u].add(v)
        if not self.directed:
            self.adj[v].add(u)
    | Explain | Add Comment |
    def remove_edge(self, u: Any, v: Any) -> bool:
        if u not in self.adj or v not in self.adj[u]:
            return False
        self.adj[u].remove(v)
        if not self.directed:
            self.adj[v].remove(u)
        return True
    | Explain | Add Comment |
    def neighbors(self, v: Any) -> Iterable[Any]:
        return iter(self.adj.get(v, set()))
    | Explain | Add Comment |
    def display(self) -> str:
        lines: List[str] = []
        for node in sorted(self.adj.keys(), key=lambda x: str(x)):
            nbrs = sorted(self.adj[node], key=lambda x: str(x))
            lines.append(f"{node}: {nbrs}")
        return "\n".join(lines)
    | Explain | Add Comment |
    def bfs(self, start: Any) -> List[Any]:
        if start not in self.adj:
            return []
        visited: Set[Any] = {start}
        order: List[Any] = []
        q: Deque[Any] = deque([start])
        while q:
            cur = q.popleft()
            order.append(cur)
            for n in sorted(self.adj[cur], key=lambda x: str(x)):
                if n not in visited:
                    visited.add(n)
                    q.append(n)
        return order
    | Explain | Add Comment |
    def dfs_recursive(self, start: Any) -> List[Any]:
        order: List[Any] = []
        visited: Set[Any] = set()
        | Explain | Add Comment |
        def _dfs(u: Any) -> None:
            visited.add(u)
            order.append(u)
            for n in sorted(self.adj.get(u, []), key=lambda x: str(x)):
                if n not in visited:
                    _dfs(n)
        if start in self.adj:
            _dfs(start)
        return order
    | Explain | Add Comment |
    def dfs_iterative(self, start: Any) -> List[Any]:
        if start not in self.adj:
            return []
        visited: Set[Any] = set()
        stack: List[Any] = [start]
        order: List[Any] = []
        while stack:
            cur = stack.pop()
            if cur in visited:
                continue
            visited.add(cur)
            order.append(cur)
            # push neighbors in reverse-sorted order to visit smallest first
            for n in sorted(self.adj.get(cur, []), key=lambda x: str(x), reverse=True):
                if n not in visited:
                    stack.append(n)
        return order
if __name__ == "__main__":
    g = Graph(directed=False)
    edges = [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E")]
    for u, v in edges:
        g.add_edge(u, v)
    print("Adjacency list:")
    print(g.display())
    print()
    print("BFS from 'A':", g.bfs("A"))
    print("DFS recursive from 'A':", g.dfs_recursive("A"))
    print("DFS iterative from 'A':", g.dfs_iterative("A"))
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/graphs.py"
Adjacency list:
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C', 'E']
E: ['D']

BFS from 'A': ['A', 'B', 'C', 'D', 'E']
DFS recursive from 'A': ['A', 'B', 'D', 'C', 'E']
DFS iterative from 'A': ['A', 'B', 'D', 'C', 'E']
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq PROMPT: Generate priority Queue using py module.

ASS_11.1 U | AI ASS LAB_11.1.py U | singly_linked_list.py U | binary_search_tree.py U | priority_queue.py U

priority_queue.py > PriorityQueue > __repr__

```python
import heapq
import itertools
from typing import Any, List, Optional, Tuple
| Explain | Add Comment |
class PriorityQueue:
    """A simple priority queue using heapq.

    Items are stored as (priority, tie_breaker, item). Lower `priority` values
    are popped first. `tie_breaker` ensures a stable order for equal priorities.
    """
    | Explain | Add Comment |
    def __init__(self) -> None:
        self._heap: List[Tuple[int, int, Any]] = []
        self._counter = itertools.count()
    | Explain | Add Comment |
    def push(self, item: Any, priority: int = 0) -> None:
        """Push `item` with numeric `priority` (lower = higher priority)."""
        heapq.heappush(self._heap, (priority, next(self._counter), item))
    | Explain | Add Comment |
    def pop(self) -> Any:
        """Remove and return the highest-priority item. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("pop from empty priority queue")
        return heapq.heappop(self._heap)[2]
    | Explain | Add Comment |
    def peek(self) -> Optional[Any]:
        """Return the highest-priority item without removing it, or None if empty."""
        return None if self.is_empty() else self._heap[0][2]
    | Explain | Add Comment |
    def is_empty(self) -> bool:
        """Return True if the queue has no items."""
        return not self._heap
    | Explain | Add Comment |
    def __len__(self) -> int:
        return len(self._heap)
    | Explain | Add Comment |
    def __repr__(self) -> str:
        items = [f"(p={p}, item={i!r})" for p, _, i in sorted(self._heap)]
        return "PriorityQueue([" + ", ".join(items) + "])"
if __name__ == "__main__":
    # Demo: lower numeric value -> higher priority
    pq = PriorityQueue()
    pq.push('low', priority=5)
    pq.push('medium', priority=3)
    pq.push('urgent', priority=1)
    pq.push('also medium', priority=3)

    print('Peek:', pq.peek())
    print('Size:', len(pq))
    while not pq.is_empty():
        print('Pop:', pq.pop())
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd6
ktop/AI ASS COD/priority_queue.py"
Peek: urgent
Size: 4
Pop: urgent
Pop: medium
Pop: also medium
Pop: low
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

PROMPT: Generate a double-ended queue in deque

ASS_11.1 U | AI ASS LAB_11.1.py U | singly_linked_list.py U | binary_search_tree.py U | priority_queue.p

deque_ds.py > Deque > __repr__

```python
from collections import deque
from typing import Any, Deque as DequeType, Iterable, List, Optional
class Deque:
    """Double-ended queue wrapper around collections.deque.

    Methods:
    - append_right(item), append_left(item)
    - pop_right(), pop_left()
    - peek_right(), peek_left()
    - is_empty(), to_list(), clear()
    """
    def __init__(self, items: Optional[Iterable[Any]] = None) -> None:
        self._dq: DequeType[Any] = deque(items) if items is not None else deque()
    def append_right(self, item: Any) -> None:
        self._dq.append(item)
    def append_left(self, item: Any) -> None:
        self._dq.appendleft(item)
    def pop_right(self) -> Any:
        if self.is_empty():
            raise IndexError("pop_right from empty deque")
        return self._dq.pop()
    def pop_left(self) -> Any:
        if self.is_empty():
            raise IndexError("pop_left from empty deque")
        return self._dq.popleft()
    def peek_right(self) -> Optional[Any]:
        return None if self.is_empty() else self._dq[-1]
    def peek_left(self) -> Optional[Any]:
        return None if self.is_empty() else self._dq[0]
    def is_empty(self) -> bool:
        return not self._dq
    def to_list(self) -> List[Any]:
        return list(self._dq)
    def clear(self) -> None:
        self._dq.clear()
    def __len__(self) -> int:
        return len(self._dq)
    def __repr__(self) -> str:
        return f"Deque({list(self._dq)})"
if __name__ == "__main__":
    d = Deque()
    print("Empty:", d.is_empty())
    d.append_right(1)
    d.append_right(2)
    d.append_left(0)
    print("After appends:", d)
    print("Peek left:", d.peek_left())
    print("Peek right:", d.peek_right())
    print("Pop left:", d.pop_left())
    print("Pop right:", d.pop_right())
    print("Remaining:", d.to_list())
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/deque_ds.py"
Empty: True
After appends: Deque([0, 1, 2])
Peek left: 0
Peek right: 2
Pop left: 0
Pop right: 2
Remaining: [1]
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation

PROMPT:USING ABOVE GENERATE ONE CODE FOR IMPLEMENT

```python
"""Library due-date manager using a priority queue for next-due lookups.

This module provides a simple `LibraryManager` that tracks borrowed books
and their due dates. The manager uses a min-heap (priority queue) keyed by
due date so the next-due book can be retrieved in O(log n) time.
"""
from __future__ import annotations
import heapq
import itertools
from datetime import date, datetime
from typing import Dict, Optional, Tuple
def _to_date(d) -> date:
    if isinstance(d, date):
        return d
    if isinstance(d, str):
        # accept YYYY-MM-DD
        return datetime.fromisoformat(d).date()
    raise TypeError("due_date must be a date or ISO date string")
class LibraryManager:
    """Manage borrowed books and provide next-due lookups.

    Internally uses:
    - a min-heap of (due_ord, tie_counter, book_id) for next-due queries
    - a dict mapping book_id -> (student_id, due_date) for fast lookup and returns

    The heap is lazily cleaned: returned books remain in the heap until popped.
    """
    def __init__(self) -> None:
        self._heap = []  # list[tuple[int, int, str]]
        self._counter = itertools.count()
        self._book_map: Dict[str, Tuple[str, date]] = {}
    def borrow_book(self, student_id: str, book_id: str, due_date) -> None:
        """Record that `student_id` borrowed `book_id` until `due_date`.

        `due_date` may be a `datetime.date` or an ISO date string `YYYY-MM-DD`.
        Raises ValueError if the `book_id` is already borrowed.
        """
        due = _to_date(due_date)
        if book_id in self._book_map:
            raise ValueError(f"book {book_id!r} already borrowed")
        self._book_map[book_id] = (student_id, due)
        heapq.heappush(self._heap, (due.toordinal(), next(self._counter), book_id))

    def return_book(self, book_id: str) -> bool:
        """Return a book. Returns True if the book was recorded and removed."""
        return self._book_map.pop(book_id, None) is not None

    def _clean_top(self) -> None:
        """Pop heap entries that are stale (book returned or moved)."""
        while self._heap:
            due_ord, _, book_id = self._heap[0]
            entry = self._book_map.get(book_id)
            if entry is None:
                heapq.heappop(self._heap)
                continue
            _, real_due = entry
            if real_due.toordinal() != due_ord:
                # due date changed (re-borrowed) or stale entry
                heapq.heappop(self._heap)
                continue
            break
    def next_due(self) -> Optional[Tuple[str, str, date]]:
        """Return (book_id, student_id, due_date) for the next due book, or None."""
        self._clean_top()
        if not self._heap:
            return None
        due_ord, _, book_id = self._heap[0]
        student_id, due = self._book_map[book_id]
        return (book_id, student_id, due)
    def pop_next_due(self) -> Optional[Tuple[str, str, date]]:
        """Remove and return the next-due book, or None if none."""
        self._clean_top()
        if not self._heap:
            return None
        _, _, book_id = heapq.heappop(self._heap)
        student_id, due = self._book_map.pop(book_id)
        return (book_id, student_id, due)
    def list_all_due(self) -> list[Tuple[str, str, date]]:
        """Return a list of all currently borrowed books sorted by due date."""
        items = [(v[1], k, v[0]) for k, v in self._book_map.items()]
        # items are (due_date, book_id, student_id)
        items_sorted = sorted(items, key=lambda t: t[0].toordinal())
        return [(book_id, student_id, due_date) for due_date, book_id, student_id in items_sorted]
if __name__ == "__main__":
    mgr = LibraryManager()
    mgr.borrow_book('s001', 'B-100', '2026-03-01')
    mgr.borrow_book('s002', 'B-200', '2026-02-28')
    mgr.borrow_book('s003', 'B-300', '2026-03-05')
    print('Next due:', mgr.next_due())
    print('All due sorted:')
    for book_id, student_id, due in mgr.list_all_due():
        print('-', book_id, student_id, due.isoformat()[
    print('\nPop next due ->', mgr.pop_next_due())
    print('After pop, next due:', mgr.next_due())
```

Output:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> ^C
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/library_due_manager.py"
Next due: ('B-200', 's002', datetime.date(2026, 2, 28))
All due sorted:
- B-200 s002 2026-02-28
- B-100 s001 2026-03-01
- B-300 s003 2026-03-05

Pop next due -> ('B-200', 's002', datetime.date(2026, 2, 28))
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #10: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products

dynamically.

2. Order Processing System – Orders processed in the order they are

placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery

locations.

PROMPT: Select suitable data structures for the given Smart E-Commerce features and justify your choices in 2–3 sentences. Then implement any one feature as a Python program with comments and docstrings.

singly_linked_list.py U | binary_search_tree.py U | priority_queue.py U | AI ASS LAB_11.1.py U

AI ASS LAB_11.1.py > ...

```python
class OrderQueue:
    """
    Order Processing System using Queue Data Structure.

    This class manages customer orders in FIFO order.
    The first order placed will be processed first.
    """

    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = []

    def place_order(self, order_id):
        """
        Add a new order to the queue.

        Args:
            order_id (int): Unique order ID
        """
        self.orders.append(order_id)
        print(f"Order {order_id} placed.")

    def process_order(self):
        """
        Process the first order in the queue.

        Returns:
            int: Processed order ID

        Raises:
            IndexError: If no orders exist.
        """
        if self.is_empty():
            print("No orders to process.")
            return None

        order = self.orders.pop(0)
        print(f"Order {order} processed.")
        return order

    def display_orders(self):
        """Display all pending orders."""
        print("Pending Orders:", self.orders)

    def is_empty(self):
        """
        Check if queue is empty.

        Returns:
            bool: True if empty, False otherwise.
        """
        return len(self.orders) == 0

```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDr
ve/Desktop/AI ASS COD/AI ASS LAB_11.1.py"
Order 101 placed.
Order 102 placed.
Order 103 placed.
Pending Orders: [101, 102, 103]
Order 101 processed.
Pending Orders: [102, 103]
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```